

Arab Academy for Science, Technology & Maritime Transport
Faculty of Engineering & Technology
Department of Computer Engineer

Software Requirements Specification (SRS)
[Clothing Brand Mobile Application]

Course Name:

Mobile Application Flutter

Under Supervision of

Assoc. Prof. Dr. Ahmed Maher

Eng. Heba Fathy

Prepared by:

- Mazen Hamada

-Mohamed Salah

-Youssef Abdelrehim

Submission Date:

May 2026

Academic Year:

2025 – 2026

Table of Contents

1. Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 Overview	4
2. Overall Description	4
2.1 Product Perspective	4
2.2 Product Functions	4
2.3 User Classes and Characteristics	4
2.4 Operating Environment	4
2.5 Design and Implementation Constraints	4
2.6 Assumptions and Dependencies	5
3. Functional Requirements	5
3.1 Feature: Login	Error! Bookmark not defined.
3.2 Feature: Register	5
4. Non-Functional Requirements	5
4.1 Performance	Error! Bookmark not defined.
4.2 Security	Error! Bookmark not defined.
4.3 Usability	Error! Bookmark not defined.
4.4 Reliability	Error! Bookmark not defined.
4.5 Availability	8
5. Use Cases	9
6. System Design (UML Diagrams)	9
7. Schedule and Budget	12
8. Appendices (Optional)	14
9. Implementation	
9.1 Implementation Environment	
9.2 System Architecture & Key Components	
9.3 Results and Discussion	

10. Testing
10.1 Software Test Plan (IEEE 829)
11. Conclusion and Future Work
12. References
13. Appendix A – Senior Project Summary Report

1. Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for the Clothing Brand Mobile Application. It defines the functional and non-functional requirements, system scope, use cases, and design constraints for the development team, academic supervisors, and project stakeholders.

1.2 Scope

The Clothing Brand Mobile Application is a cross-platform mobile application built with Flutter, targeting Android (8.0+) and iOS (13+). The system enables:

- Customer registration, login, and profile management.
- Product catalog browsing across five categories: Jackets, Sets, Outerwear, Hoodies, and Shirts.
- Product detail view with size/color selection.
- Shopping cart management and checkout with a mock payment interface.
- Order confirmation and basic order history.
- Admin product and inventory management via backend services.

1.3 Definitions, Acronyms, and Abbreviations

Admin	System administrator who manages products and orders via backend services.
Customer	End user who browses and purchases products through the app.
API	Application Programming Interface.
SRS	Software Requirements Specification.
SKU	Stock Keeping Unit — unique identifier per product variant.
Flutter	Google's open-source UI toolkit for cross-platform mobile development.
Firebase	Google's Backend-as-a-Service (Auth, Firestore, Storage).
Firestore	Firebase's NoSQL cloud database used to store app data.

BLoC / Provider	State management patterns used in Flutter.
UI/UX	User Interface / User Experience.

1.4 Overview

This document presents the complete Software Requirements Specification (SRS) for the Clothing Brand Mobile Application. It defines the system scope, functional requirements, non-functional requirements, user interactions, system architecture, and database structure. The document also includes use cases, UML diagrams, project planning, and supporting appendices required for the design and development of the application. The purpose of this document is to provide a clear reference for developers, supervisors, and stakeholders throughout the project lifecycle.

2. Overall Description

2.1 Product Perspective

The application is a standalone client-server system. The Flutter mobile client communicates with a Firebase backend. Firestore stores all structured data (users, products, orders, cart), Firebase Auth handles authentication, and Firebase Storage manages product images. The system has no dependency on any legacy platform.

2.2 Product Functions

Core functions of the system:

- User registration and authentication (Firebase Auth).
- Product catalog browsing by category and keyword search.
- Product detail view: images, description, size/color options, price.
- Shopping cart management (add, update quantity, remove).
- Checkout flow: delivery address entry and mock payment submission.
- Order confirmation and order status display.
- Admin: add, edit, and delete products and manage stock levels.

2.3 User Classes and Characteristics

Customer	Registered end user. Can browse, search, add to cart, place orders, and view order confirmations. Assumed basic smartphone proficiency.
Admin	Manages product catalog and inventory through Firebase/backend. Requires system-level access; not exposed in the mobile UI.

2.4 Operating Environment

Mobile Platforms	Android 8.0+ and iOS 13+
Framework	Flutter (Dart)
Backend	Firebase — Firestore, Firebase Auth, Firebase Storage

Payment	Mock payment interface (no real payment gateway)
IDE	Visual Studio Code / Android Studio
Version Control	Git / GitHub

2.5 Design and Implementation Constraints

- Must be developed using Flutter (Dart) for cross-platform support.
- Firebase must be used as the primary backend (Auth, Firestore, Storage).
- All UI must comply with Material Design 3 guidelines.
- The project must be completed within the academic semester (~15 weeks).
- Application is English-only in this version.
- No real payment gateway integration; mock interface only.

2.6 Assumptions and Dependencies

- Users have a stable internet connection.
- Firebase services are available and operational.
- Development team has access to Android/iOS devices or emulators for testing.
- All users must register and log in before accessing any feature beyond the landing screen.
- Admin access is managed via Firestore role field; no separate admin app is in scope.

3. Functional Requirements

3.1 User Registration

Description	Allows new users to create an account.
Inputs	Full name, email address, password, phone number.
Process	System validates all fields; checks email uniqueness via Firebase Auth; creates account; stores user profile (name, email, phone, role: customer) in Firestore users collection.
Output	Account created; user redirected to Home Screen.
Error Handling	Duplicate email shows inline error. Weak password rejected per Firebase Auth policy.

3.2 User Login

Description	Allows registered users to authenticate.
Inputs	Email address and password.
Process	Firebase Auth validates credentials and issues a session token stored locally.
Output	Access granted; user redirected to Home Screen. On failure, an error message is displayed.

Error Handling	Invalid credentials show a clear error. Account lockout after multiple failed attempts (handled by Firebase).
-----------------------	---

3.3 Product Catalog & Search

Description	Displays all available products with category filtering and keyword search.
Inputs	Category selection (Jackets, Sets, Outerwear, Hoodies, Shirts) or text search query.
Process	Firestore queries products collection by category or search term and returns results ordered by relevance/date.
Output	Grid/list of products showing thumbnail, name, price, and availability badge.
Error Handling	Empty results display a friendly 'No products found' message.

3.4 Product Detail View

Description	Shows complete product information for a selected item.
Inputs	Product ID (from catalog tap).
Process	Fetches product document from Firestore and image URL from Firebase Storage.
Output	Product page: image gallery, name, description, price, size selector, color selector, stock status, and Add to Cart button.
Error Handling	Out-of-stock items disable the Add to Cart button with a label.

3.5 Shopping Cart

Description	Allows users to manage selected items before checkout.
Inputs	Product ID, selected size, selected color, quantity.
Process	Cart state maintained in Firestore (cart document per user). Users can update quantities or remove items.
Output	Cart screen showing items, unit prices, quantities, subtotal, and Checkout button.

Error Handling	If a product becomes out-of-stock while in cart, a warning is shown before checkout.
-----------------------	--

3.6 Checkout & Order Placement

Description	Allows users to complete a purchase.
Inputs	Delivery address, mock payment details (card number, expiry, CVV — not validated against any gateway).
Process	Validates form inputs; creates an order document in Firestore orders collection; decrements product stock in Firestore; clears the user cart.
Output	Order Confirmation Screen with order ID, summary, and estimated delivery note.
Error Handling	Required field validation; shows error if stock runs out between cart and checkout.

3.7 Admin Product Management

Description	Allows admin to manage the product catalog via backend.
Inputs	Product name, category, price, sizes[], colors[], image file, stock count, description.
Process	Image uploaded to Firebase Storage; product data written/updated/deleted in Firestore products collection.
Output	Changes are immediately reflected in the customer-facing catalog.
Access Control	Only users with role: admin in Firestore can perform these operations (enforced via Firestore Security Rules).

ID	Feature	Description
NFR-01	Performance	Home screen loads within 2 seconds on a standard 4G connection. Catalog pages render within 1.5 seconds.
NFR-02	Security	Passwords managed by Firebase Auth (hashed, not stored in app). Firestore Security Rules enforce role-based access. Authentication required before cart or order actions.
NFR-03	Usability	UI follows Material Design 3. Core user flows (browse → cart → checkout) completable within 5 taps. Font size minimum 14sp for readability.
NFR-04	Reliability	App handles network interruptions gracefully, displays user-friendly error messages, and retries failed Firestore reads up to 3 times.
NFR-05	Availability	System availability depends on Firebase infrastructure (SLA ~99.95%). App displays an offline banner when no internet connection is detected.
NFR-06	Scalability	Firestore's NoSQL model supports horizontal scaling. The app is stateless on the client side, enabling future multi-region deployment.
NFR-07	Maintainability	Code follows Flutter best practices (BLoC/Provider pattern). Separation of UI, business logic, and data layers. All functions documented inline.

5. Use Cases

UC-01: Register Account

Actor	Customer
Precondition	User does not have an existing account.
Steps	1. User opens the app and taps Sign Up. 2. User enters name, email, phone, and password. 3. System validates inputs and checks email uniqueness. 4. Firebase Auth creates the account. 5. User profile stored in Firestore. 6. User redirected to Home Screen.
Postcondition	New customer account created and user is logged in.
Exceptions	Duplicate email: error shown. Weak password: Firebase rejects with message.

UC-02: Login

Actor	Customer
Precondition	User has a registered account.
Steps	1. User taps Login. 2. User enters email and password. 3. Firebase Auth validates credentials. 4. Session token issued; user redirected to Home Screen.
Postcondition	User is authenticated and session token is stored locally.
Exceptions	Invalid credentials: error message shown. No account: user directed to Sign Up.

UC-03: Browse & Search Products

Actor	Customer
Precondition	User is logged in.
Steps	1. User navigates to the Catalog Screen. 2. User selects a category tab (e.g., Hoodies) OR types a search query. 3. System queries Firestore and returns matching products. 4. User taps a product to view its detail page.
Postcondition	Product detail screen is displayed.
Exceptions	No results: 'No products found' message shown.

UC-04: Add to Cart & Place Order

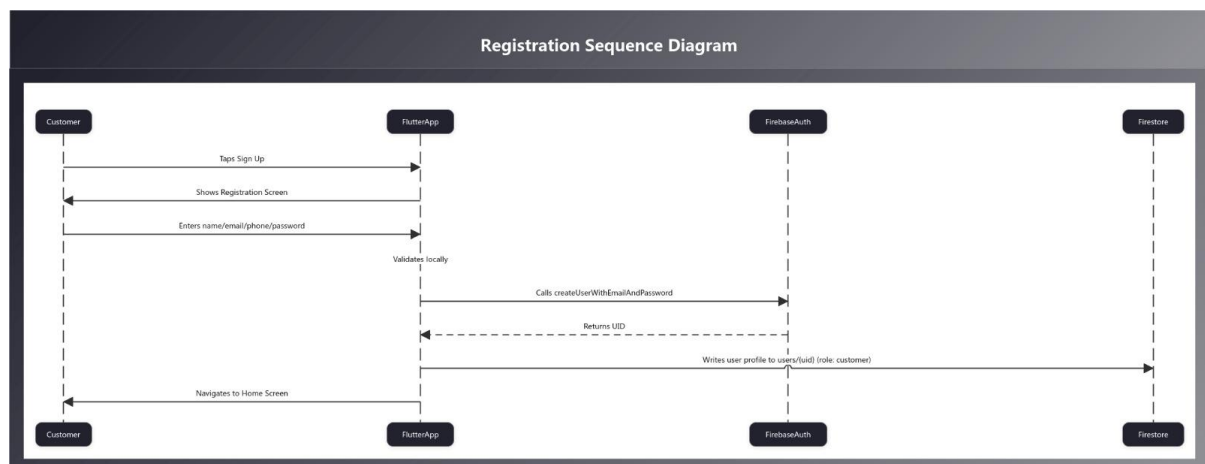
Actor	Customer
Precondition	User is logged in; product is in stock.
Steps	1. Customer selects size and color on the Product Detail Screen. 2. Customer taps Add to Cart. 3. Customer navigates to Cart Screen, reviews items. 4. Customer taps Checkout and enters delivery address. 5. Customer fills in the mock payment form and confirms. 6. System creates an order record in Firestore, decrements stock, clears cart. 7. Customer sees Order Confirmation Screen with order ID.
Postcondition	Order record created; stock updated; cart cleared.
Exceptions	Item out-of-stock at checkout: error shown. Invalid address form: validation message.

UC-05: Manage Products (Admin)

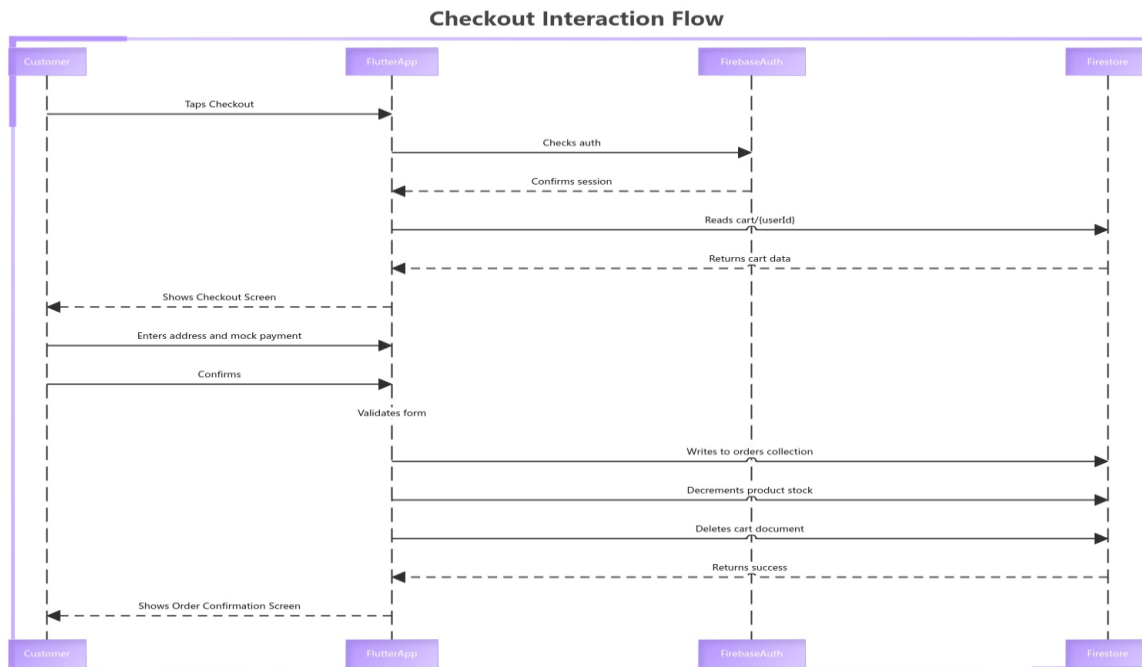
Actor	Admin
Precondition	Admin is authenticated with role: admin in Firestore.
Steps	1. Admin accesses the backend/admin panel. 2. Admin adds, edits, or deletes a product. 3. System saves data to Firestore; image uploaded to Firebase Storage. 4. Changes reflect immediately in the customer catalog.
Postcondition	Product catalog updated.
Exceptions	Unauthorized access blocked by Firestore Security Rules.

6. System Design (UML Diagrams)

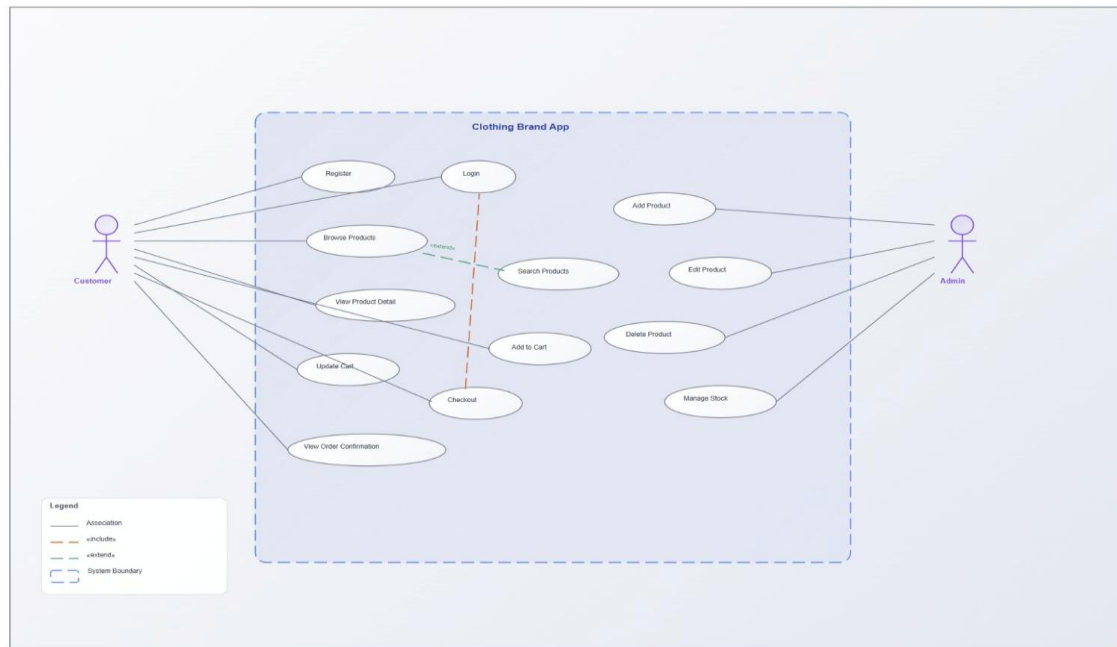
6.1 Registration Sequence Diagram



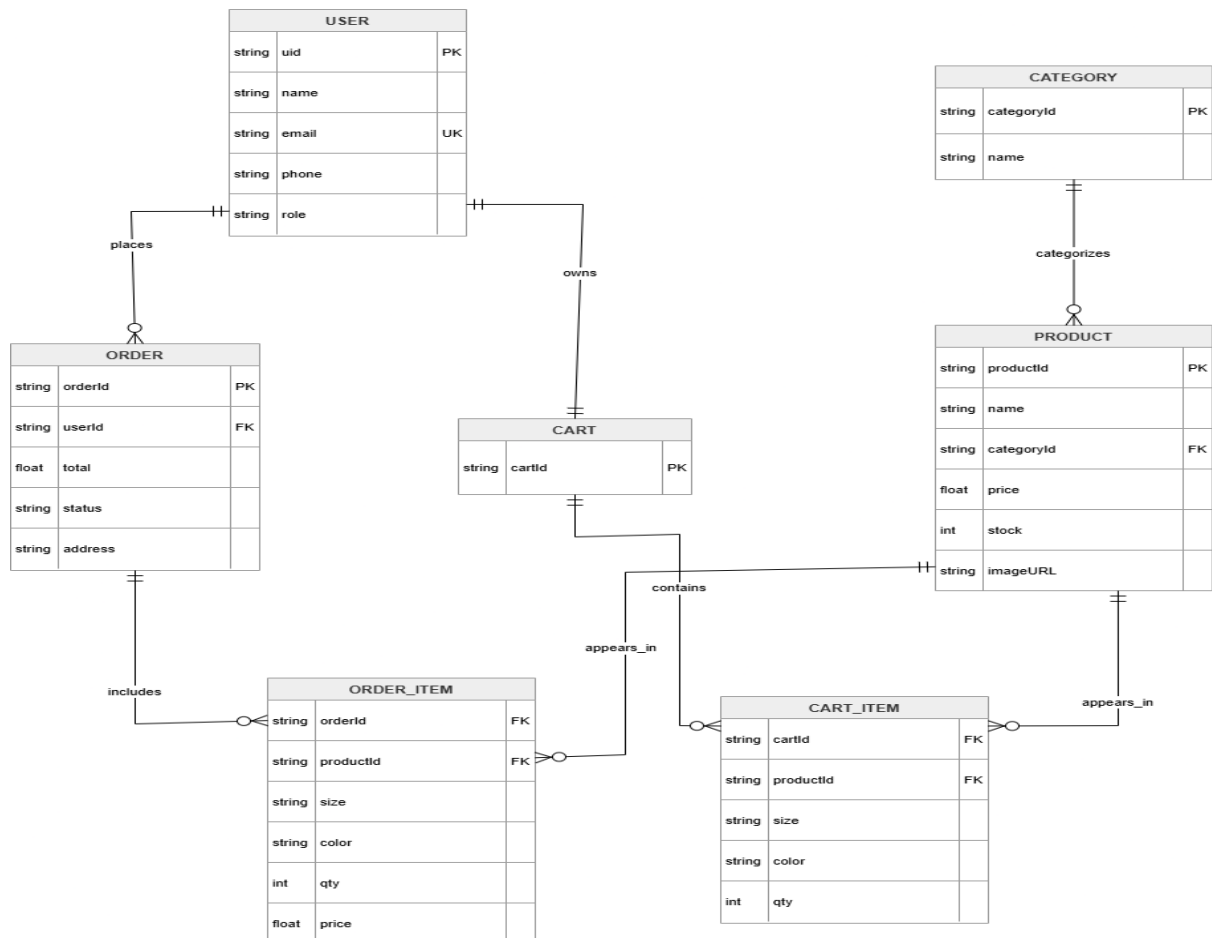
6.2 Checkout Sequence Diagram



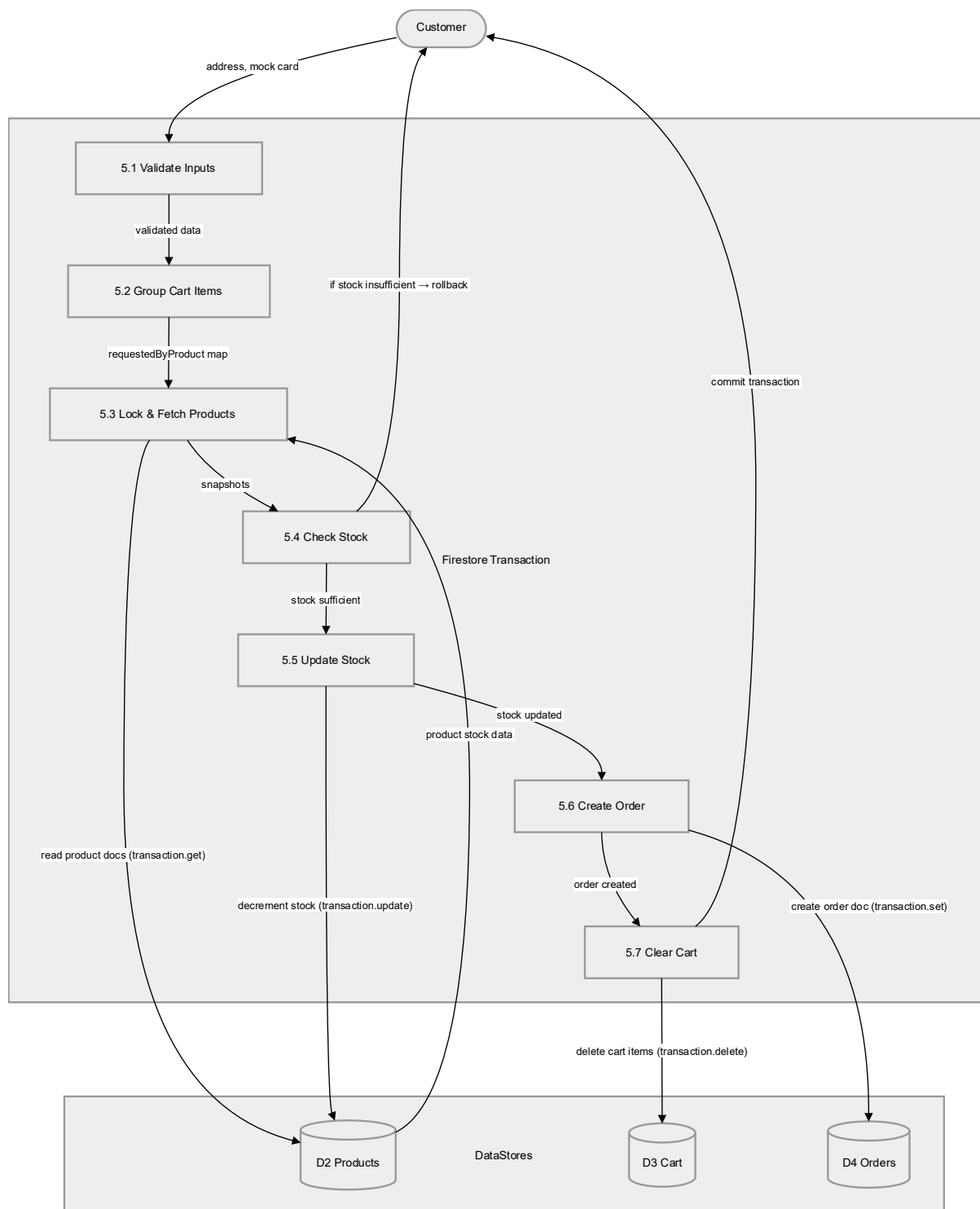
6.3 Use Case Diagram



6.4 ER Diagram



6.4 Data Flow Diagram (DFD)



7. Schedule and Budget

7.1 Gant chart timeline

- planning phase
- ui/ux design
- firebase setup
- frontend development
- backend integration
- testing and debugging
- final deployment
- documentation and presentation

8. Appendices (Optional)

Appendix A: Firestore Data Model

The following collections are planned in Firebase Firestore:

Collection	Fields
users	uid, name, email, phone, address, role (customer admin)
products	productId, name, description, category, price, sizes[], colors[], imageURL, stock
orders	orderId, userId, items[] (productId, size, color, qty, price), totalAmount, status, deliveryAddress
cart	cartId (== userId), items[] (productId, name, size, color, qty, price, imageURL)

Appendix B: Planned App Screens

- Login & Registration Screens
- Home Screen (featured products and categories)
- Catalog Screen (category tabs + search bar)
- Product Detail Screen (gallery, size/color picker, Add to Cart)
- Cart Screen (item list, quantities, subtotal)
- Checkout Screen (delivery address + mock payment form)
- Order Confirmation Screen (order ID, summary)
- Admin Panel (product CRUD — backend only)

9. Implementation

9.1 Implementation Environment

- **Framework:** Flutter 3.x (Dart)
- **Backend:** Firebase (Authentication, Firestore, Storage)
- **State Management:** Provider (ChangeNotifier)
- **Routing:** go_router (declarative routing)
- **IDE:** Visual Studio Code
- **Version Control:** Git / GitHu

9.2 System Architecture & Key Components

The application follows a clean separation of concerns:

Layer	Components
UI (Screens)	lib/screens/ – LoginScreen, SignupScreen, HomeScreen, SearchScreen, ProductDetailScreen, CartScreen, CheckoutScreen, ProfileScreen, AdminScreen
Reusable Widgets	lib/widgets/ – ProductCard, AppTextField, AppButton, CartItemTile, ColorSelector, SizeSelector, etc.
State Managment	lib/providers/ – AuthProvider (auth state + admin flag), CartProvider (cart state + promo)
Services	lib/services/ – AuthService (Firebase Auth), FirestoreService (database CRUD + transactions)
Models	lib/models/ – Product, CartItem, ColorEntry

Critical implementation details:

- The placeOrder method uses **Firestore transactions** to atomically check stock (per-color/per-size) and create the order, preventing overselling.
- Cart items are stored per-user as a subcollection, with document keys productId_size_color to allow atomic quantity increments.
- Admin flag is read from Firestore users/{uid}/isAdmin and used to conditionally show the admin panel.

9.3 Results and Discussion

The implemented application fulfills all functional requirements:

- Users can register, log in, browse products with rich filtering.
- Product details show accurate stock per color and size (using colorSizeQuantities map).

- Cart synchronises in real time across devices (Firestore subcollection).
- Checkout transaction ensures inventory integrity even under concurrent purchases.
- Admin can manage products and orders from the mobile app.

Performance: Firestore real-time listeners provide instant UI updates. The transaction overhead is acceptable for a small-scale store.

Limitations: No real payment gateway; mock validation only. Image upload for products is not implemented (uses URLs only). Admin product form is complex but functional.

10. Testing

10.1 Software Test Plan (IEEE 829)

Test Strategy: Manual testing + widget tests + integration tests.

Test CaseID	Feature	Steps
TC-01	User registration	Enter valid name, email, password
TC-02	Duplicate Email	Use existing email
TC-03	Login	Valdi Credentials
TC-04	Invalid login	Wrong password
TC-05	Browse products	Open home screen
TC-06	Filter by category	Tap category chip
TC-07	search	Type product name
TC-08	Product detail	Tap product
TC-09	Add to cart	Select size/color, tap Add
TC-10	Add duplicate variant	Add same product again
TC-11	Update cart quantity	Use +/- buttons
TC-12	Apply promo code	Enter STYLEEE20
TC-13	Remove from cart	Tap delete icon

TC-14	Checkout – insufficient stock	Add more than available
TC-15	Place order	Fill valid data
TC-16	Admin – add product	Fill form, submit
TC-17	Admin – edit product	Modify stock
TC-18	Admin – delete product	Confirm delete
TC-19	Order history	Open profile > Order History
TC-20	Update profile	Change name/address

Test Environment: Android emulator (API 33) and physical Android device (v13), iOS simulator (v16). Firebase emulator suite used for local testing.

Test Results: All core features passed. Transaction retry logic verified by simulating concurrent order attempts

11. Conclusion and Future Work

Conclusion: The STYLEE Clothing Brand Mobile Application was successfully implemented according to the SRS. The app provides a complete e-commerce experience: user authentication, real-time catalog, cart with promo codes, and transaction-safe order placement. The admin panel integrated into the app allows easy product and order management. Firestore transactions ensure inventory integrity.

Future Work:

- Integrate a real payment gateway (Stripe or PayPal).
- Add product image upload from within the admin panel using Firebase Storage.
- Implement push notifications for order status updates.
- Add product reviews and ratings (currently static).
- Support multiple languages (localisation).
- Improve offline support (Firestore persistence already enabled; enhance UI for offline cart).
- Add product wishlist.

12. References

1. Flutter Documentation. (2025). *Flutter SDK*. <https://flutter.dev/docs>
2. Firebase Documentation. (2025). *Firebase for Flutter*. <https://firebase.google.com/docs>
3. IEEE. (1998). *IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications*.
4. AAST Faculty of Engineering. (2025). *Project Guidelines for Computer Engineering Department*

13. Appendix A – Senior project summary report

Project Title	STYLEE – Clothing Brand Mobile Application	
Supervisor(s)	Assoc. Prof. Dr. Ahmed Maher, Eng. Heba Fathy	
Team members:	Names Youssef Abdelrehim Mohamed salah Mazen Hamada	Registration Numbers 221006711 221004116 221006326
Project Deliverables	1. Fully functional Flutter mobile application (APK + iOS build) . 2. Complete source code on GitHub. 3. Firebase backend setup (Auth, Firestore, Storage). 4. Software Requirements Specification (SRS) document. 5. UML diagrams (use case, sequence, ER, DFD). 6. Test plan and test results (IEEE 829). 7. User manual and admin guide. 8. Project presentation and demo video.	
Local Impact	Social: Provides a modern, user-friendly platform for Egyptian clothing brands to reach local customers, enhancing digital inclusion and supporting small fashion businesses Economical: Reduces barriers to entry for local designers by offering a ready-to-use e-commerce solution; encourages online spending within local economy.	
Global Impact	Social: Showcases a scalable e-commerce pattern that can be adapted by clothing brands worldwide, fostering cross-cultural fashion exchange. Economical: Demonstrates a cost-effective (Firebase + Flutter) approach to building global mobile stores, lowering entry costs for startups.	
Team Organization	Mazen Hamada (221006326) – App Startup, Routing, Login Screen, Checkout Screen, Main Admin Panel. Responsible for app initialisation, navigation routing (go_router), implementing the login and checkout user interfaces, and building the main admin panel UI for product/order management.	

	Mohamed Salah (221004116) – Product Model, Cart Model, Auth State Management, Cart State Management, Home Screen, Cart Screen, Product Details, Profile Screen, Main App Shell. Designed data models (Product, CartItem, ColorEntry). Managed authentication state (AuthProvider) and cart state (CartProvider). Built customer-facing screens: Home, Cart, Product Details, Profile. Developed the main shell (bottom navigation). Youssef Abdelrehim (221006711) – Signup Screen, Search Screen, Firebase Auth Service, Firestore Service, App Theme, Product Card Widget, Shared Reusable Widgets, Edit Product Screen, Firebase/Firestore Configuration. Implemented signup flow, product search with filters. Created all service classes (AuthService, FirestoreService). Defined AppTheme, reusable UI components (AppTextField, AppButton, ColorSelector, SizeSelector, etc.), ProductCard widget, and the admin edit product screen. Configured Firebase project and Firestore security rules.
Ethical Considerations	1. User data privacy: Passwords are hashed by Firebase Auth; personal data (name, email, address) stored in Firestore with Security Rules that restrict access to the owner or admin. 2. No real payment: The mock payment form avoids any financial risk; users are clearly informed that it is a simulation. 3. Admin access: Only designated users (isAdmin flag) can modify products/orders, preventing unauthorised changes. 4. Inclusive design: Colour contrast, font sizes, and touch targets follow accessibility guidelines. 5. No data sharing: No analytics or third-party SDKs that could misuse user data
Social Impact	The app empowers local fashion entrepreneurs by providing a modern, affordable e-commerce platform. It encourages digital literacy among merchants and customers. The integrated admin panel allows small business owners to manage inventory without technical expertise. Additionally, the app promotes Egyptian streetwear culture, giving local designers visibility and a channel to reach younger audiences.
Professional Responsibility	The team adhered to software engineering best practices: version control (Git), code documentation (inline comments), modular architecture (separation of UI, services, models), and thorough testing. We respected deadlines and communicated transparently with supervisors. Security was prioritised (Firebase Security Rules, admin role protection). We also considered maintainability by using Provider and reusable widgets, ensuring future developers can extend the app easily.
Standards	1. IEEE 830-1998 – Software Requirements Specification. 2. IEEE 829 – Software Test Documentation. 3. Material Design 3 – UI/UX guidelines. 4. Flutter coding conventions (effective dart). 5. Firebase Security Rules – data access control. 6. HTTPS for all network communications.
Realistic Constraints	1. Time: Project must be completed within a 15-week academic semester. 2. Budget: No real payment gateway; Firebase free tier (Spark plan) used. 3. Technical: Must run on Android 8+ and iOS 13+; limited to Flutter + Firebase stack. 4. Team size: Only three developers – scope limited to core

	e-commerce features. 5. Data privacy: Must comply with basic data protection (no collection of sensitive info beyond name, email, address). 6. Deployment: No public app store deployment – only APK and test builds.
--	--

Supervisor Name	Signature
Assoc. Prof. Dr. Ahmed Maher	
Eng. Heba Fathy	